

# **Group 8 Invoice System**

## **System Design Document**

---

**Version 1.0**

**10/06/2026**

# Table of Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                                   | <b>3</b>  |
| 1.1 Purpose of the SDD .....                                   | 3         |
| <b>2 General Overview and Design Guidelines/Approach .....</b> | <b>4</b>  |
| 2.1 General Overview .....                                     | 4         |
| 2.2 Assumptions/Constraints/Risks .....                        | 4         |
| Assumptions.....                                               | 5         |
| Constraints .....                                              | 5         |
| Risks   5                                                      |           |
| <b>3 Design Considerations .....</b>                           | <b>6</b>  |
| 3.1 Development Methods .....                                  | 6         |
| 3.2 Architectural Strategies .....                             | 6         |
| Design decisions .....                                         | 6         |
| Data Objects .....                                             | 6         |
| 3.3 Entity-Relationship Diagram .....                          | 6         |
| <b>4 System Architecture and Architecture Design .....</b>     | <b>8</b>  |
| 4.1 Component Diagram.....                                     | 8         |
| 4.2 Web Architecture .....                                     | 9         |
| 4.3 Software Architecture.....                                 | 9         |
| Separation of concerns .....                                   | 9         |
| AI integration.....                                            | 9         |
| Cohesion and coupling.....                                     | 9         |
| Human oversight and improvement lifecycle.....                 | 9         |
| 4.4 System Architecture Component Diagram.....                 | 1         |
| 4.5 Logical Diagram.....                                       | 1         |
| 4.6 Interface Detailed Design.....                             | 1         |
| <b>5 System Design .....</b>                                   | <b>5</b>  |
| 5.1 Business Requirements .....                                | 5         |
| 5.2 Database Design .....                                      | 7         |
| 5.3 User Interface .....                                       | 9         |
| Inputs   10                                                    |           |
| Outputs 11                                                     |           |
| <b>6 Operational Scenarios .....</b>                           | <b>13</b> |
| 6.1 Scenario 1 – Successful Upload .....                       | 13        |
| 6.2 Scenario 2 – Later viewing.....                            | 13        |
| 6.3 Error handling .....                                       | 13        |

# **1. Introduction**

---

The SDD documents the high-level system design and the low-level detailed design specifications. The SDD describes design goals and considerations, provides a high-level overview of the system architecture, and describes the data design associated with the system, as well as the human-machine interface and operational scenarios. The high-level system design is further decomposed into low-level detailed design specifications for each system component, including hardware, internal communications, software, system integrity controls, and external interfaces.

## **1.1 Purpose of the SDD**

The SDD documents and tracks the necessary information required to effectively define architecture and system design in order to ensure the project team all have the same expectations on the architecture of the system to be developed.

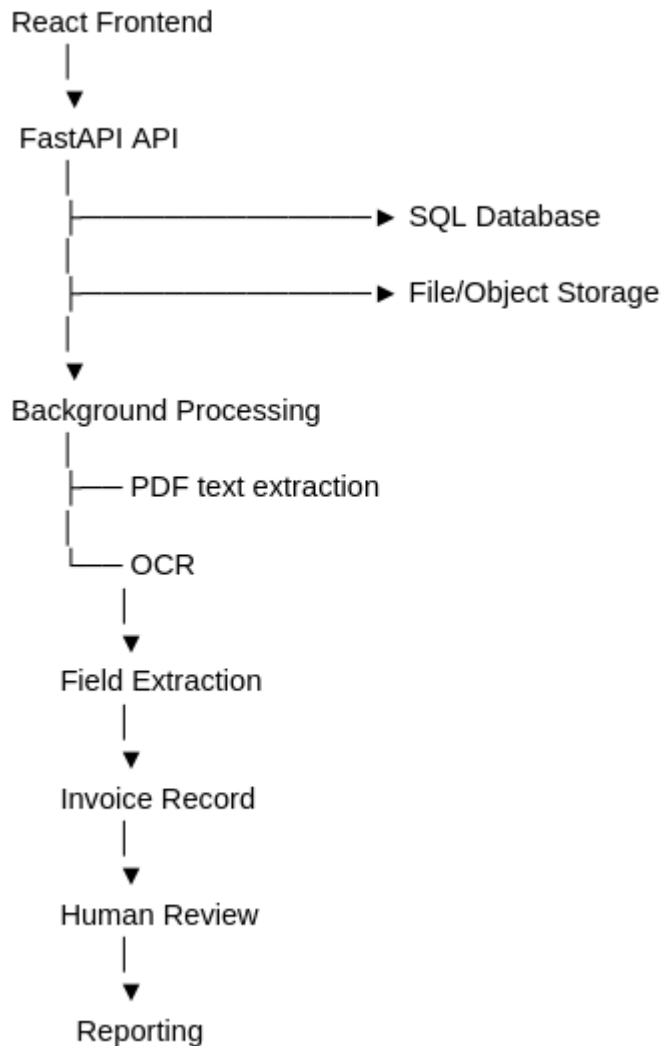
## 2 General Overview and Design Guidelines/Approach

This section describes the principles and strategies to be used as guidelines when designing and implementing the system.

### 2.1 General Overview

Our project will address the time and errors involved in manually processing workplace invoices. Employees often have to read invoices and enter vendor names, invoice numbers, dates, line items, and totals into company systems in order to process database updates and generate reports. We propose a system that automates much of this process while keeping a person involved in verifying the results.

The architecture will be a 3-tier architecture as follows:



### 2.2 Assumptions/Constraints/Risks

## Assumptions

- Users upload invoice PDFs through the React UI.
- PDFs may be text-based or scanned/image-based.
- OCR extracts key fields such as vendor, invoice number, date, subtotal, tax, total, and currency.
- Extracted data is reviewed/corrected by a user before being committed.
- FastAPI provides the API and orchestrates processing.
- SQL stores invoices and extracted fields.
- Reporting initially means a searchable/filterable invoice report with CSV export.
- Authentication, complex workflow/approval, and advanced analytics are out of scope for the first 8 weeks.

## Constraints

- The software must be publicly available for testing purposes
- The hosting must be free

## Risks

The three main technical risks we see are OCR errors, sensitive invoice data being exposed, and integration failures between our components. We plan to mitigate these through human verification/confidence thresholds, authentication and encryption, and API validation, error handling, logging, and integration testing.

## 3 Design Considerations

---

### 3.1 Development Methods

For the tech stack, we planned to use React for the frontend, Python/FastAPI for the backend, PostgreSQL for storing invoice data, and an existing Optical Character Recognition (OCR) solution, Python Tesseract or a library based off of it such as PyOCR, for document processing. Bednarz and Milosz (2025) benchmarked Python web frameworks and found FastAPI performed strongly across API tests. Chen et al. (2025) evaluated OCR technologies on 122 invoices and demonstrated high extraction proficiency in Tesseract.

### 3.2 Architectural Strategies

#### Design decisions

One decision was whether to allow the Python backend to perform multiple steps of the invoice upload at once. It was possible to use one Python function to update the invoice, line item, and inventory databases in sequence. We elected to call these methods from the React frontend in three separate calls to Main in order to provide better user feedback when one step should fail.

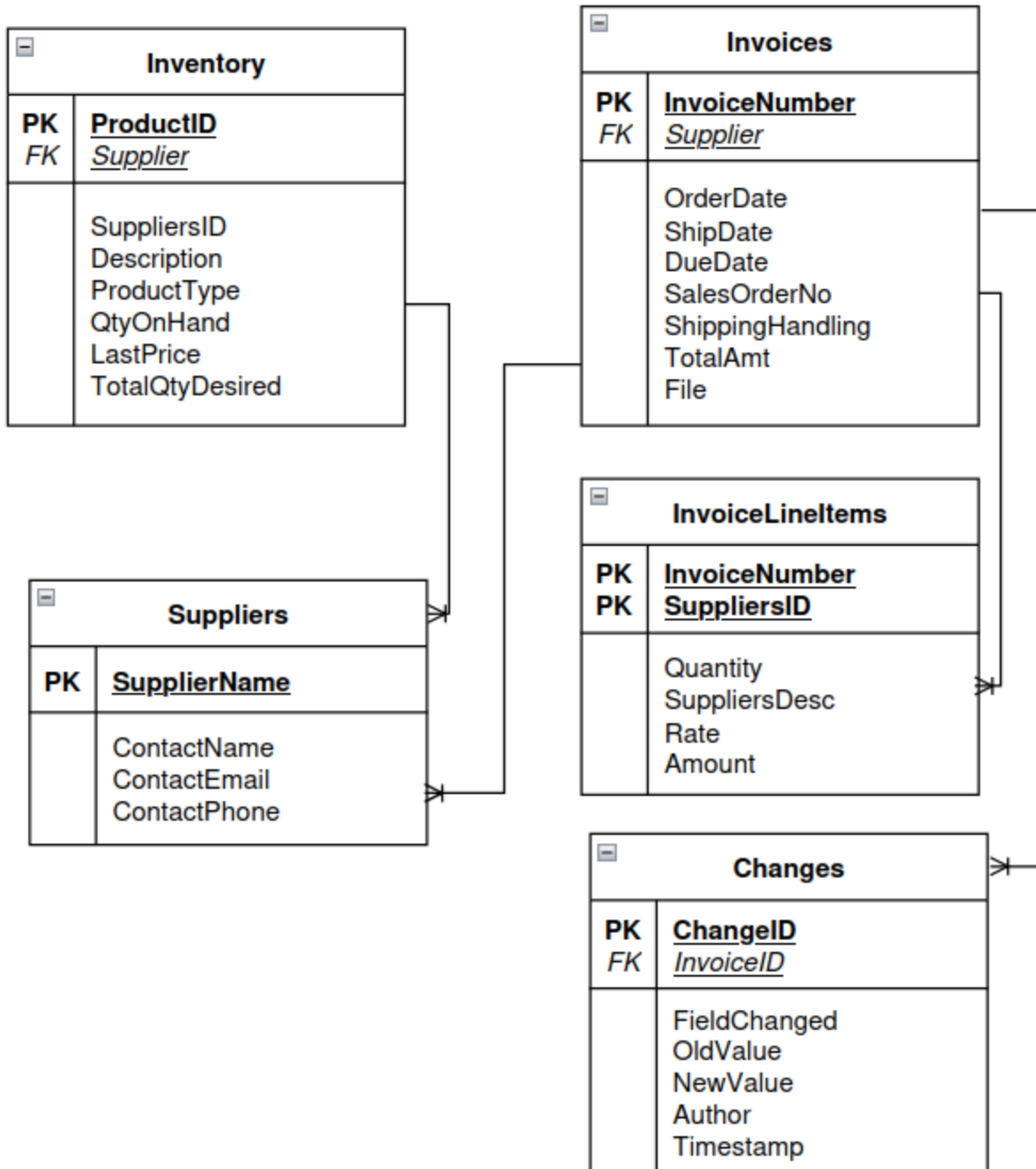
SQLite was used instead of a cloud connection to a SQL database since it reduces the network attack surface because the SQLite database isn't exposed as a separate remote database service and the invoice data stays on the application host. Reduced external was considered a security benefit.

For document processing, we used PyMuPDF to read PDFs and EasyOCR, an open-source OCR library, to recognize text in scanned invoices. We chose EasyOCR over Tesseract mainly for its text detection and its fit with our deployment. In a real-world evaluation of 1,628 product-label images, we found EasyOCR more accurate on digits, which dominate invoice fields. EasyOCR also installs through pip without a separate system dependency, which suits our free-hosting constraint, uses pretrained models that need no training data, and returns a confidence score for each detected text region, which populates the OCRConfidence field.

#### Data Objects

Invoices are stored in FastAPI memory for the purpose of performing text extraction, and then on the user's hard drive.

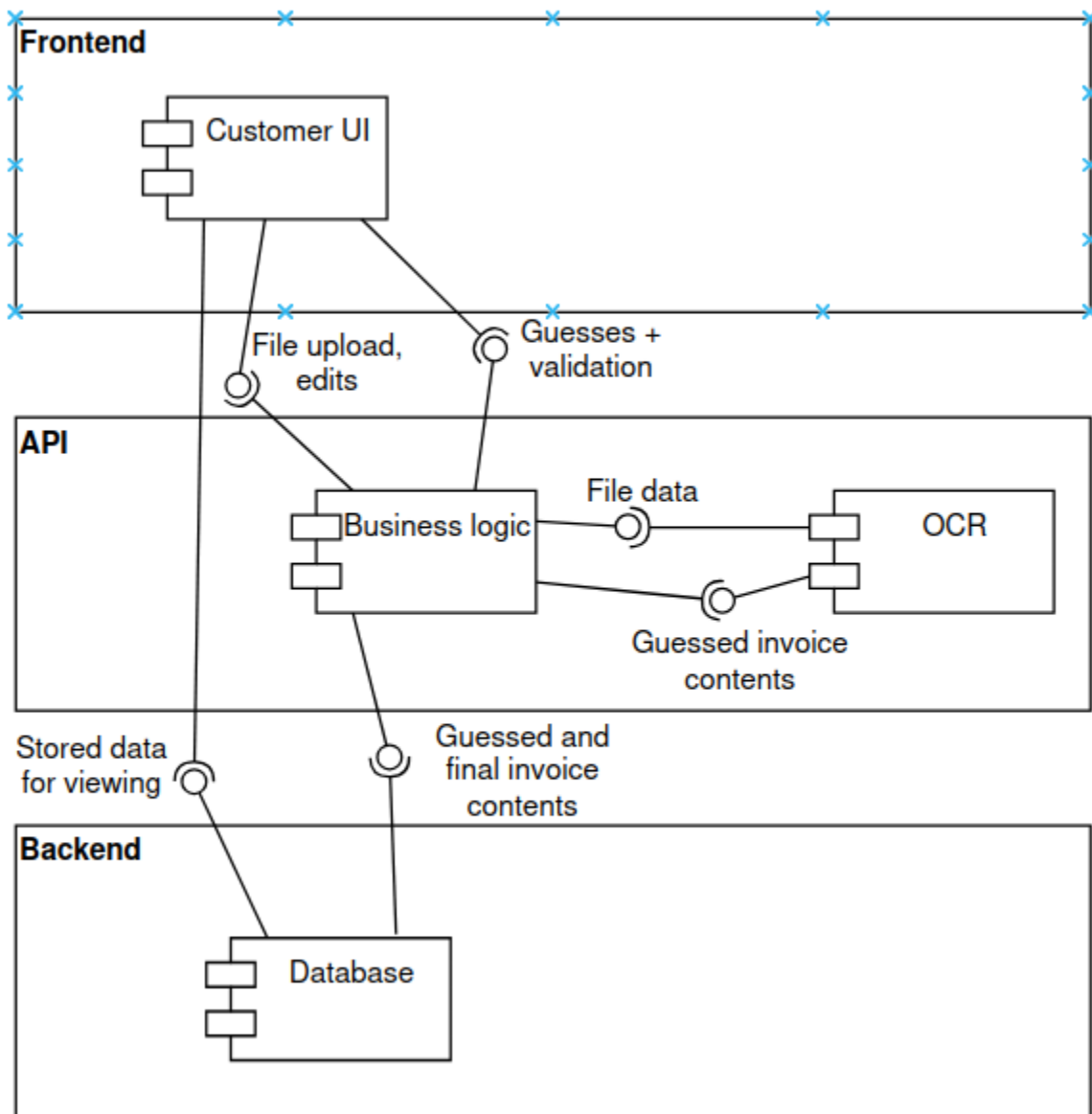
### 3.3 Entity-Relationship Diagram



## 4 System Architecture and Architecture Design

This section outlines the system and hardware architecture design of the system.

### 4.1 Component Diagram



## 4.2 Web Architecture

The system is a web-based application. Servers used include the backend database at port 8001 and the frontend at port 5173.

## 4.3 Software Architecture

### Separation of concerns

The system uses a three-layer architecture. The presentation tier (React) handles upload, review and editing, lists, and reports. The application tier (FastAPI) holds the uploader, the OCR extraction module, the validation module, and the inventory update logic. The data tier (SQL database) stores invoices, invoice line items, suppliers, inventory, and change records. Each tier has a distinct responsibility, and the tiers communicate only through defined interfaces.

### AI integration

The only AI component in the system is the invoice text-extraction (OCR) module. We implemented it as an in-process module of the FastAPI application tier rather than as a separate service, for three reasons. The system is small, so one backend keeps it simple to build and test. Our constraint of free hosting limits us to a single backend deployment. A separate service would also add network calls, failure points, and deployment effort without a clear benefit at our scale. If the module later needs to scale independently, the JSON contract makes it straightforward to move behind its own API.

### Cohesion and coupling

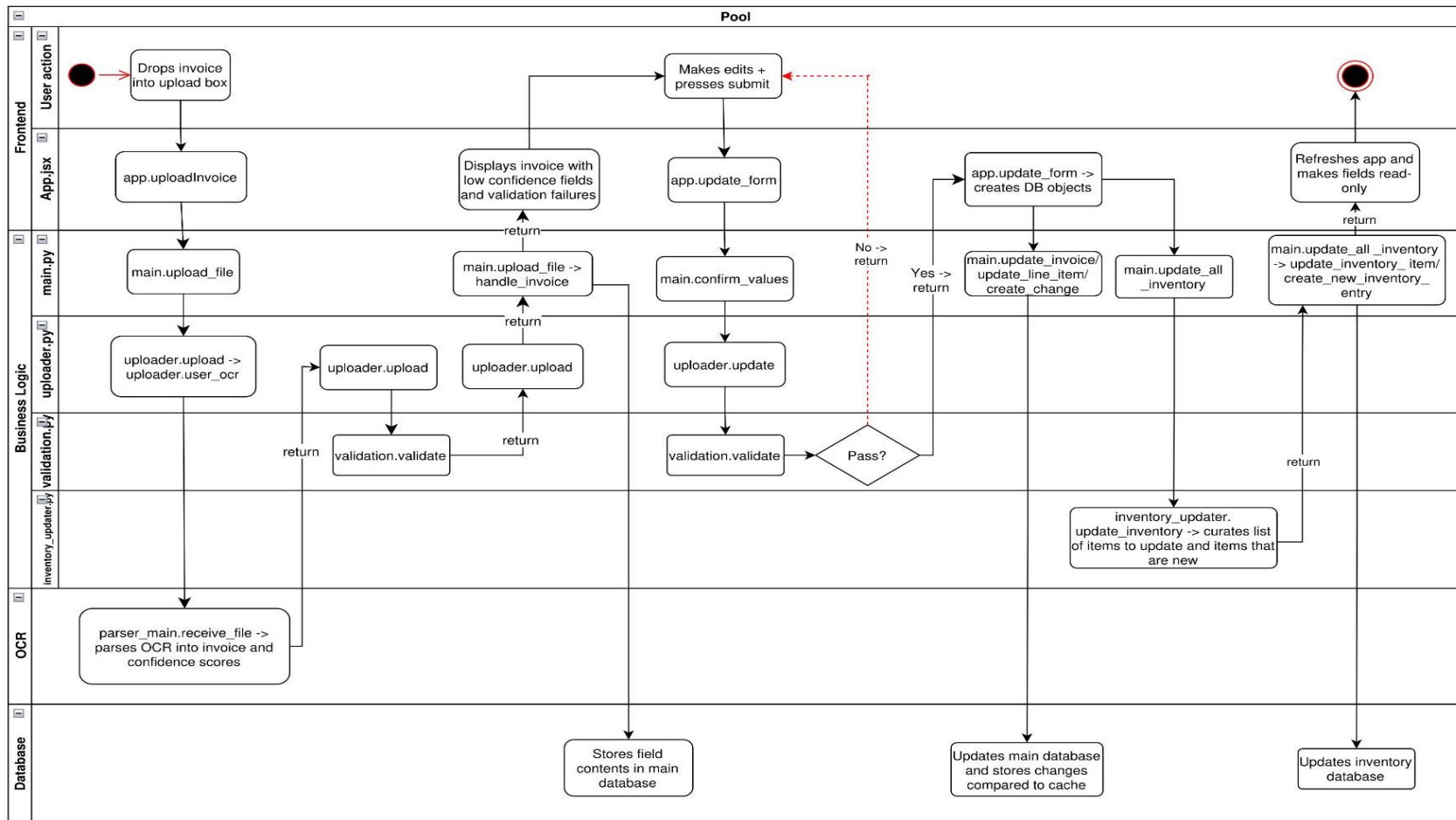
Each module in the application tier has one job. The extraction module turns a document into text and structured fields. The validation module enforces required-field, amount, and date rules. The uploader coordinates the two: it calls extraction, receives its data as a JSON object, parses it into application schema objects, and calls validation on the initial upload and on every edit. The extraction module does not read or write the database and does not know about validation rules.

The modules are loosely coupled through a fixed JSON contract. The OCR engine can be replaced without changing the rest of the system as long as the replacement returns the same schema. Between tiers, coupling is limited to the REST API: the frontend never accesses the database or the OCR module directly, so the database schema or the OCR engine can change without breaking the user interface.

### Human oversight and improvement lifecycle

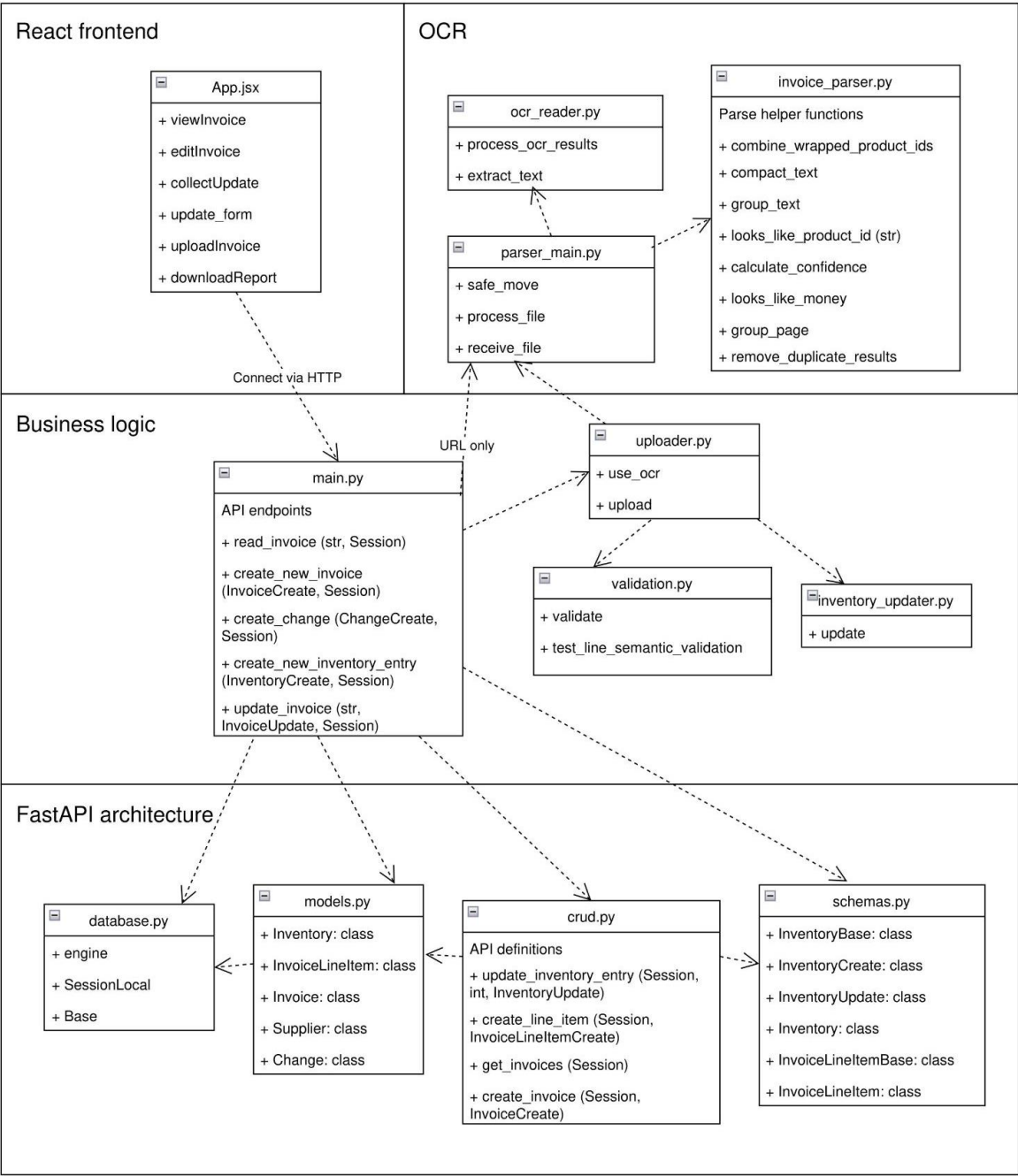
OCR output is treated as a suggestion, not a final result. Nothing is committed until a user confirms the values. Every user correction is stored as a Change record. Our system uses no prompts, no knowledge base, and no model is retrained, because EasyOCR is used as a pretrained model. User corrections therefore never change the model itself. Instead, the team periodically reviews Change records for repeated errors and adjusts the field-mapping rules, the confidence threshold, or the image preprocessing. Each such change is made through a pull request, reviewed by a second team member, and must pass the CI workflows before it is merged.

#### 4.4 System Architecture Component Diagram



One thing that caused some confusion was oversimplification of the workflow, since the loose coupling meant several classes acted mostly as controllers to interface with other classes. Therefore, the only place where the exact flow of information has been simplified is the calls to databases. Methods that call databases do wait for a "success" response before continuing.

### 4.5 Logical Diagram



### 4.6 Interface Detailed Design

The complete API contracts and all object schemas can be found at <http://127.0.0.1:8001/docs> once the backend is running locally. For baselining and reference purposes, a printout copy is included with this document.

For ease of understanding this document, a **summary** is provided below.

| Method / Endpoint                       | Purpose                    | Main Input                                 | Main Output                     |
|-----------------------------------------|----------------------------|--------------------------------------------|---------------------------------|
| GET /invoices/all                       | Read invoices              | none                                       | List of Invoice objects         |
| POST /invoices/all                      | Create new invoice         | InvoiceCreate object                       | InvoiceCreate object            |
| GET /inventory                          | Read inventory             | none                                       | List of Inventory objects       |
| POST /inventory                         | Create new inventory entry | InventoryCreate object                     | InventoryCreate object          |
| GET /suppliers                          | Read suppliers             | none                                       | List of Supplier objects        |
| GET /changes                            | Read all changes           | None                                       | List of Change objects          |
| GET /invoices/{InvoiceNumber}           | Read invoice               | String InvoiceNumber                       | Invoice object                  |
| PUT /invoices/{InvoiceNumber}           | Update invoice             | String InvoiceNumber, InvoiceUpdate object | InvoiceUpdate object            |
| GET /invoices/{InvoiceNumber}/lineitems | Read invoice items         | String InvoiceNumber                       | List of InvoiceLineItem objects |

| Method / Endpoint                                     | Purpose                                                   | Main Input                                                  | Main Output                                                 |
|-------------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------|
| POST<br>/invoices/{InvoiceNumber}/lineitems           | Create new line item                                      | String InvoiceNumber,<br>InvoiceLineItemCreate object       | InvoiceLineItemCreate object                                |
| GET /inventory/{ProductID}                            | Read inventory entry                                      | String ProductID                                            | Inventory object                                            |
| PUT /inventory/{ProductID}                            | Update inventory entry                                    | String ProductID,<br>InventoryUpdate object                 | InventoryUpdate object                                      |
| GET /invoices/{InvoiceNumber}/changes                 | Read changes for a given invoice                          | String InvoiceNumber                                        | List of Change objects                                      |
| POST /invoices/{InvoiceNumber}/changes                | Create change                                             | ChangeCreate object                                         | ChangeCreate object                                         |
| GET /invoices/{InvoiceNumber}/changes/{ChangeID}      | Read change                                               | Integer ChangeID                                            | Change object                                               |
| PUT /invoices/{InvoiceNumber}/lineitems/{SuppliersID} | Update line item                                          | String InvoiceNumber,<br>String SuppliersID                 | InvoiceLineItem object                                      |
| POST /confirm_values                                  | Collect user-edited invoice and pass to validation module | InvoiceCreate object, list of InvoiceLineItemUpdate objects | InvoiceCreate object, list of InvoiceLineItemUpdate objects |
| POST /update_all_inventory                            | Update all inventory objects mentioned by invoice         | InvoiceCreate object, list of InvoiceLineItemUpdate objects | InvoiceCreate object, list of InvoiceLineItemUpdate objects |
| POST /upload                                          | Collects file from user                                   | UploadFile object                                           | Invoice object, list of InvoiceLineItem objects, list of    |

| Method / Endpoint | Purpose                                    | Main Input | Main Output          |
|-------------------|--------------------------------------------|------------|----------------------|
|                   |                                            |            | confidence intervals |
| GET /download     | Prompts the report to compile and download | none       | String               |

The parser output schema, while not an object, is as follows:

```
{
  "Invoice":
    {
      "InvoiceNumber": str,
      "OrderDate": str,
      "ShipDate": str,
      "DueDate": str,
      "ShippingHandling": float, (negative if not supplied)
      "TotalAmt": float, (negative if not supplied)
      "Supplier": str,
    }
  "InvoiceLineItems": [{
    "SuppliersID": str,
    "Description": str,
    "Quantity": int, (theoretically)
    "HSCode": str,
    "Rate": float,
    "Amount": float,
    "OCRConfidence": float,
    "Confidence": float,
    "LineCount": int,
  }]
}
```

## **5 System Design**

---

### **5.1 Business Requirements**

| Priority | Backlog item                               | Acceptance criteria                                                                                                     |
|----------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| P0       | Define invoice data model                  | Required fields, types, validation rules, and invoice lifecycle documented                                              |
| P0       | Set up SQL database and migrations         | Local/dev DB works; schema created through migrations                                                                   |
| P0       | Create FastAPI project structure           | Health endpoint, configuration, logging, error handling                                                                 |
| P0       | Create React application shell             | Routing, layout, API client, basic styling                                                                              |
| P0       | Define OCR extraction contract             | OCR output mapped to normalized invoice fields                                                                          |
| P0       | Build PDF upload API                       | Valid PDF can be uploaded; invalid file extensions rejected; upload receives unique ID                                  |
| P0       | Build invoice upload UI                    | User can select/upload PDF                                                                                              |
| P0       | Implement PDF text extraction              | Text-based PDFs produce usable text                                                                                     |
| P0       | Integrate OCR engine                       | Scanned invoice can be converted into machine-readable text                                                             |
| P1       | Store original PDF and processing metadata | File reference, timestamps, status, and errors are persisted                                                            |
| P0       | Implement invoice field extraction         | Vendor, invoice #, dates, amounts, tax, total, currency extracted where present                                         |
| P1       | Add confidence scores                      | Extracted fields have confidence/quality indicators                                                                     |
| P0       | Create invoice detail API                  | UI can retrieve extracted invoice                                                                                       |
| P0       | Build invoice review screen                | User sees PDF/extracted fields and can edit fields                                                                      |
| P0       | Save corrected invoice data                | User edits persist to SQL with validation                                                                               |
| P0       | Add invoice validation                     | Required fields and amount/date rules are enforced                                                                      |
| P1       | Handle OCR failures                        | Failed invoices receive meaningful status/error and can be retried                                                      |
| P0       | Build invoice list                         | Invoice list with vendor, invoice #, date, total                                                                        |
| P2       | Build supplier list                        | Supplier list with contact information                                                                                  |
| P1       | Build inventory list                       | Paginated inventory list that hides items neither had nor wanted                                                        |
| P1       | Add search/filtering                       | Search vendor/invoice # and filter by date/status. Filter inventory by product type, quantity needed, quantity on hand. |

|    |                                   |                                                             |
|----|-----------------------------------|-------------------------------------------------------------|
| P1 | Implement duplicate detection     | System identifies likely duplicate invoice numbers/vendors  |
| P0 | Build report endpoint             | API returns invoice data suitable for reporting             |
| P0 | Build report UI                   | Users can view totals and invoice records                   |
| P1 | Add CSV export                    | Filtered report can be downloaded as CSV                    |
| P1 | Add basic report totals           | Total invoices, subtotal, tax, and invoice amount displayed |
| P1 | Add field-level review indicators | Low-confidence fields are clearly highlighted               |
| P1 | Add retry/reprocess controls      | Authorized user can retry failed OCR processing             |

## 5.2 Database Design

Entities, attributes, and keys are described in the tables following.

### Inventory

| # | Name                   | Type          | Collation         | Attributes | Null | Default     | Comments | Extra |
|---|------------------------|---------------|-------------------|------------|------|-------------|----------|-------|
| 1 | <b>ProductID</b>       | varchar(10)   | latin1_swedish_ci |            | No   | <i>None</i> |          |       |
| 2 | <b>SuppliersID</b>     | varchar(50)   | latin1_swedish_ci |            | Yes  | <i>NULL</i> |          |       |
| 3 | <b>Description</b>     | varchar(120)  | latin1_swedish_ci |            | No   | <i>None</i> |          |       |
| 4 | <b>ProductType</b>     | varchar(20)   | latin1_swedish_ci |            | No   | <i>None</i> |          |       |
| 5 | <b>QtyOnHand</b>       | int(10)       |                   | UNSIGNED   | No   | 0           |          |       |
| 6 | <b>LastPrice</b>       | decimal(10,2) |                   |            | Yes  | <i>NULL</i> |          |       |
| 7 | <b>TotalQtyDesired</b> | int(11)       |                   |            | Yes  | <i>NULL</i> |          |       |
| 8 | <b>Supplier</b>        | varchar(50)   | latin1_swedish_ci |            | Yes  | <i>NULL</i> |          |       |

### Indexes

| Keyname        | Type  | Unique | Packed | Column    | Cardinality | Collation | Null | Comment |
|----------------|-------|--------|--------|-----------|-------------|-----------|------|---------|
| <b>PRIMARY</b> | BTREE | Yes    | No     | ProductID | 663         | A         | No   |         |

## InvoiceLineItems

| # | Name          | Type          | Collation         | Attributes | Null | Default | Comments | Extra |
|---|---------------|---------------|-------------------|------------|------|---------|----------|-------|
| 1 | InvoiceNumber | varchar(10)   | latin1_swedish_ci |            | No   | None    |          |       |
| 2 | Quantity      | int(11)       |                   |            | No   | None    |          |       |
| 3 | SuppliersID   | varchar(50)   | latin1_swedish_ci |            | No   | None    |          |       |
| 4 | SuppliersDesc | text          | latin1_swedish_ci |            | No   | None    |          |       |
| 5 | Rate          | decimal(10,2) |                   |            | No   | None    |          |       |
| 6 | Amount        | decimal(10,2) |                   |            | No   | None    |          |       |
| 7 | LineCount     | int(11)       |                   |            | Yes  | NULL    |          |       |

### Indexes

| Keyname | Type  | Unique | Packed | Column        | Cardinality | Collation | Null | Comment |
|---------|-------|--------|--------|---------------|-------------|-----------|------|---------|
| PRIMARY | BTREE | Yes    | No     | InvoiceNumber | 2           | A         | No   |         |
|         |       |        |        | SuppliersID   | 39          | A         | No   |         |

## Invoices

| # | Name             | Type        | Collation         | Attributes | Null | Default | Comments | Extra |
|---|------------------|-------------|-------------------|------------|------|---------|----------|-------|
| 1 | InvoiceNumber    | varchar(50) | latin1_swedish_ci |            | No   | None    |          |       |
| 2 | OrderDate        | date        |                   |            | No   | None    |          |       |
| 3 | ShipDate         | date        |                   |            | No   | None    |          |       |
| 4 | DueDate          | date        |                   |            | No   | None    |          |       |
| 5 | SalesOrderNo     | varchar(50) | latin1_swedish_ci |            | No   | None    |          |       |
| 6 | ShippingHandling | float       |                   |            | No   | None    |          |       |
| 7 | TotalAmt         | float       |                   |            | No   | None    |          |       |
| 8 | Supplier         | varchar(20) | latin1_swedish_ci |            | No   | None    |          |       |
| 9 | File             | longblob    |                   |            | Yes  | NULL    |          |       |

### Indexes

| Keyname | Type  | Unique | Packed | Column        | Cardinality | Collation | Null | Comment |
|---------|-------|--------|--------|---------------|-------------|-----------|------|---------|
| PRIMARY | BTREE | Yes    | No     | InvoiceNumber | 1           | A         | No   |         |

## Changes

| # | Name                | Type         | Collation         | Attributes | Null | Default                    | Comments | Extra          |
|---|---------------------|--------------|-------------------|------------|------|----------------------------|----------|----------------|
| 1 | <b>InvoiceID</b>    | varchar(50)  | latin1_swedish_ci |            | No   | <i>None</i>                |          |                |
| 2 | <b>FieldChanged</b> | varchar(20)  | latin1_swedish_ci |            | No   | <i>None</i>                |          |                |
| 3 | <b>OldValue</b>     | varchar(50)  | latin1_swedish_ci |            | No   | <i>None</i>                |          |                |
| 4 | <b>NewValue</b>     | varchar(50)  | latin1_swedish_ci |            | No   | <i>None</i>                |          |                |
| 5 | <b>Author</b>       | varchar(120) | latin1_swedish_ci |            | Yes  | <i>NULL</i>                |          |                |
| 6 | <b>Timestamp</b>    | timestamp    |                   |            | Yes  | <i>current_timestamp()</i> |          |                |
| 7 | <b>ChangeID</b>     | int(11)      |                   |            | No   | <i>None</i>                |          | AUTO_INCREMENT |

### Indexes

| Keyname        | Type  | Unique | Packed | Column   | Cardinality | Collation | Null | Comment |
|----------------|-------|--------|--------|----------|-------------|-----------|------|---------|
| <b>PRIMARY</b> | BTREE | Yes    | No     | ChangeID | 0           | A         | No   |         |

## 5.3 User Interface

In keeping with the nature of React, the app is a single-page app. It always displays the list of invoices and the invoice upload panel. When an invoice is selected with the View button, its details appear, both metadata and the list of line items. When the Edit button is selected, fields become editable and confidence scores and validation warnings appear. Wireframes follow.

CAPSTONE GROUP 8

# Invoice Processing System

Human-in-the-loop review of workplace invoice data.

## Upload Invoice

Browse...

No file selected.

Upload PDF

## Invoices

Download Report

| Invoice     | Supplier           | Order Date | Total   |                 |
|-------------|--------------------|------------|---------|-----------------|
| INV-DEMO-01 | Bullseye Glass Co. | 2026-09-01 | \$78.25 | <div>View</div> |

## Invoice INV-DEMO-01

Edit

Supplier: Bullseye Glass Co.

Order Date: 2026-09-01

Sales Order: SO-DEMO-01

Due: 2026-10-01

Shipped On: 2026-09-02

Shipping: \$10.00

Total: \$78.25

### Line Items

|                                                                               |                       |
|-------------------------------------------------------------------------------|-----------------------|
| <div>Opaque White Opal Fine Frit 1 lb jar</div> <div>000013-0001-F-P001</div> | 1 × \$12.42 = \$12.42 |
| <div>Red Full Double-Rolled</div> <div>000124-0030-F-FULL</div>               | 1 × \$55.83 = \$55.83 |

Inputs

Supplier: Bullseye Glass Co.0.89

Order Date: 06/12/20240.95

Sales Order: S0056230.05

Due: 07/12/20240.9

Shipped On: 06/11/20240.78

Shipping and Handling: 541.360.04

Total: 5954.10.56

Submit

Line Items

Reactive Cloud Opalescent,  
000009-0030-F-FULL

1

×

\$ 62.60

= \$ 62.60

0.6

Tomato Red Opalescent, Po  
000024-0008-F-POOS

1

×

\$ 56.08

= \$ 56.08

0.53

Metadata and line items in edit mode

0.58

French Vanilla Opalescent; I  
000137-0030-F-FULL

2

×

\$ 48.82

= \$ 97.64

Description contains the word Invoice.

A line item with a validation warning

Outputs

A report may be generated that summarizes the invoices.

 report\_2026-10-06 15:52:33.txt ×

Report for 2026-10-06 15:52:33

## Invoice Summary

| Invoice Number | Supplier           | Due Date   | Total |
|----------------|--------------------|------------|-------|
| INV-DEMO-01    | Bullseye Glass Co. | 2026-10-01 | 78.25 |

## Invoice Breakdown

Invoice number: INV-DEMO-01  
Supplier: Bullseye Glass Co.  
Order date: 2026-09-01  
Ship date: 2026-09-02  
Due date: 2026-10-01  
Total: 78.25

## Line items:

#1 000013-0001-F-P001  
Description: Opaque White Opal Fine Frit 1 lb jar  
Quantity: 1  
Rate: 12.42  
Amount: 12.42

#2 000124-0030-F-FULL  
Description: Red Full Double-Rolled  
Quantity: 1  
Rate: 55.83  
Amount: 55.83

End of report

## 6 Operational Scenarios

---

### 6.1 Scenario 1 – Successful Upload

1. The accountant clicks the browse button, selects a file, and clicks the upload button.
2. The API receives the request and uploads the file, passing it to the parser.
3. The parser reads the text, formats the result into an invoice, checks for validation, and sends the information back.
4. The system displays the invoice in Edit mode with confidence scores and validation warnings.
5. The accountant makes edits and clicks Submit.
6. The system checks for validation and saves the data.

### 6.2 Scenario 2 – Later viewing

1. The accountant clicks the View button, which displays the invoice in read-only mode.
2. The invoice displays in Edit mode without confidence scores or validation warnings.
3. The accountant makes edits and clicks Submit.
4. The system checks for validation and saves the data.

### 6.3 Error handling

- Should the validation check fail, the validation errors appear, the save does not complete, and the form fields do not return to read-only.
- Should the invoice upload or any other API calls fail, a message is displayed to the user.

## Appendix A: Record of Changes

Table 1 - Record of Changes

| Version Number | Date     | Author/Owner | Description of Change                         |
|----------------|----------|--------------|-----------------------------------------------|
| 0.1            | 9/3/26   | SL           | First pass                                    |
| 0.2            | 9/4/26   | SL           | Added diagrams                                |
| 0.3            | 10/5/26  | SL           | Added API summary                             |
| 0.4            | 10/6/26  | AOS          | Added AI discussion                           |
| 1.0            | 10/6//26 | SL           | Updated diagrams and removed explanatory text |